SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 – PART II

Name: A. Abhishek Reddy

Rg no:192424192

Course Code: CSA0601

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

1. Introduction

Algorithm engineering is the process of designing, implementing, testing, debugging and improving algorithms to solve real-world computational problems. An algorithm may be mathematically correct, but it must also work efficiently with different inputs and satisfy practical requirements.

This assignment focuses on Dynamic Programming, Greedy Algorithms and Graph Algorithms. These techniques are used to solve optimisation problems in document formatting, logistics, network routing, manufacturing and delivery management.

Dynamic Programming solves complex problems by dividing them into smaller overlapping subproblems and storing their results. It avoids repeated calculations and builds an optimal solution from previously solved states.

Greedy Algorithms construct a solution by making the best available local decision at each step. They are often simple and fast, but a locally best decision does not always produce a globally optimal solution.

Graph algorithms are used to solve network-related problems such as shortest paths, minimum spanning trees and reachability between locations.

2. Objectives of the Assignment

1. Implement Dynamic Programming algorithms and understand how their states are constructed.
2. Compare Greedy and Dynamic Programming solutions using practical datasets.
3. Implement shortest-path algorithms and investigate negative-weight edges and negative cycles.
4. Develop an assembly-line scheduling system using Dynamic Programming.
5. Compare Kruskal's, Prim's and Borůvka's algorithms for minimum spanning trees.
6. Implement the Travelling Salesperson Problem using Dynamic Programming.
7. Test algorithms using normal, boundary, modified and failure cases.
8. Analyse computational work, memory usage, solution quality and scalability.

3. Engineering Workflow

The assignment follows the engineering process:

Stage	Description
Design	Understand the problem and select a suitable algorithm
Implement	Convert the algorithm into a working program
Instrument	Display intermediate states and computational work
Test	Execute the program using different inputs
Break	Modify inputs to expose possible weaknesses
Debug	Identify and correct implementation defects
Validate	Compare the result with an independent or expected result
Defend	Explain why the algorithm works and when it may fail

A successful compilation does not prove that an algorithm is correct. The result must also be tested, validated and explained.

FUNDAMENTAL CONCEPTS OF OPTIMISATION

4. Dynamic Programming

Dynamic Programming is an optimisation technique used when a problem contains overlapping subproblems and optimal substructure.

Overlapping subproblems means that the same smaller problems are solved multiple times during a direct recursive approach.

Optimal substructure means that an optimal solution to a larger problem can be constructed using optimal solutions to smaller problems.

The main steps involved in Dynamic Programming are:

1. Identify the smaller subproblems.
2. Define the DP state.
3. Establish the base condition.
4. Formulate the recurrence relation.
5. Store previously calculated results.
6. Reconstruct the final optimal solution.

There are two common implementation approaches:

- Memoization: A top-down recursive method that stores previously calculated results.

- Tabulation: A bottom-up method that fills a table from smaller states to larger states.

5. Greedy Algorithm

A Greedy Algorithm makes the best available choice at every step without reconsidering previous decisions.

The main steps are:

1. Identify the available choices.
2. Define a selection rule.
3. Select the best available choice.
4. Check whether the choice satisfies the constraints.
5. Continue until the solution is complete.

Greedy algorithms can be efficient, but their correctness depends on the problem. Fractional Knapsack can be solved optimally using value-to-weight ratio, whereas the same strategy is not guaranteed to solve 0/1 Knapsack optimally.

6. Comparison of Dynamic Programming and Greedy Strategies

Feature	**Dynamic Programming**	**Greedy Algorithm**
Decision method	Compares possible subproblem solutions	Selects the best immediate choice
Stored information	DP states and previous results	Usually selected choices and current state
Repeated computation	Avoided using stored states	Generally does not require DP tables
Optimality	Optimal when the recurrence correctly models the problem	Depends on the greedy-choice property
Memory	May require additional storage	Often requires less memory
Example	0/1 Knapsack	Fractional Knapsack

7. Importance of Instrumentation

Instrumentation means adding counters and intermediate-state displays to a program to understand how an algorithm behaves.

Useful measurements include:

- Number of DP states evaluated.
- Number of comparisons.
- Number of recursive calls.

- Number of selected items.
- Memory used by the DP table.
- Order of vertex finalisation.
- Sequence of selected edges.

Instrumentation helps explain not only the final answer but also how the algorithm reached that answer.

TASK 1 – BUILD AND VALIDATE A DYNAMIC PROGRAMMING SOLUTION

WORD WRAP PROBLEM

8. Problem Description

A document-processing system must arrange a sequence of words into lines of maximum width \(W\). Each line can contain one or more complete words, provided that the total number of characters and spaces does not exceed the given width.

Unused spaces at the end of a non-final line create a penalty. The penalty is calculated by squaring the number of unused spaces.

[text{Penalty}=s^2]

where \(s\) represents the number of unused spaces at the end of a line.

The final line does not carry any penalty.

The objective is to arrange all words into lines so that the total penalty is minimum.

8.1 Input

- A sequence of words.
- Maximum line width \(W\).

8.2 Output

The program must display:

1. Minimum total penalty.
2. Words placed on each line.
3. Number of unused spaces on each line.
4. Penalty contributed by each line.
5. Intermediate DP states used to construct the optimal arrangement.

8.3 Example Input

Words:

This is an example of dynamic programming

Maximum line width:

W = 10

The words must remain in their original order. A word cannot be divided into separate parts.

8.4 Mathematical Model

Let the words be:

[w_1,w_2,ldots,w_n]

Let (L(i,j)) represent the total width required to place words (i) through (j) on a single line.

It includes the lengths of all words and one space between consecutive words.

[L(i,j)=sum_{k=i}^{j}|w_k|+(j-i)]

If:

[L(i,j)>W]

the words cannot fit on the same line.

The unused spaces are:

[s=W-L(i,j)]

The penalty for a non-final line is:

[C(i,j)=s^2]

For a final line:

[C(i,j)=0]

This mathematical model converts document formatting into an optimisation problem.

WORD WRAP DP DESIGN AND PSEUDOCODE

9. Dynamic Programming State

Let:

[DP[i]]

represent the minimum penalty required to arrange words from position \(i\) to the final word.

The base condition is:

[DP[n]=0]

This means that no additional penalty is required when all words have already been arranged.

For each starting position \(i\), the algorithm tries every possible ending position \(j\) that fits within the line width.

The recurrence is:

[DP[i]=min_{ileq j<n} left(C(i,j)+DP[j+1]right)]

Only valid lines satisfying (L(i,j)leq W) are considered.

The algorithm stores the best ending position for each state. This information is used to reconstruct the final arrangement.

9.1 Meaning of the DP State

DP State	Meaning
(DP[0])	Minimum penalty for arranging all words
(DP[1])	Minimum penalty from the second word onwards
(DP[2])	Minimum penalty from the third word onwards
(DP[i])	Minimum penalty from word \(i\) onwards
(DP[n])	No words remain; penalty is zero

9.2 Word Wrap Pseudocode

ALGORITHM WordWrap(words, W)

INPUT:

List of words

Maximum line width W

OUTPUT:

Minimum penalty

Optimal line arrangement

```
BEGIN
  n ← number of words
  DP[n] ← 0
  FOR i ← n-1 DOWNTO 0 DO
    DP[i] ← INFINITY
    FOR j ← i TO n-1 DO
      Calculate line width from i to j
      IF line width > W THEN
        BREAK
      END IF
      unused ← W - line width
      IF j = n-1 THEN
        penalty ← 0
```

```
        ELSE
            penalty ← unused × unused
        END IF
        cost ← penalty + DP[j+1]
        IF cost < DP[i] THEN
            DP[i] ← cost
            next[i] ← j+1
        END IF
    END FOR
  END FOR
  Reconstruct lines using next[]
  RETURN DP[0] and line arrangement
END
```

9.3 Why the Algorithm Uses Dynamic Programming

The same remaining sequence of words may be considered from different earlier line decisions. Storing \(DP[i]\) avoids solving the same remaining formatting problem repeatedly.

The algorithm also checks different possible line endings. This helps it select a globally minimum-penalty arrangement instead of choosing a line based only on immediate unused spaces.

WORD WRAP IMPLEMENTATION AND VALIDATION

10. Illustrative Word Wrap Arrangement

Consider the following input:

Parameter	Value
Words	This, is, an, example, of, dynamic, programming
Maximum width	10

One possible valid arrangement is:

Line	Words	Used width	Unused spaces	Penalty
1	This is an	9	1	1
2	example of	10	0	0
3	dynamic	7	3	0

4	programming	11	—	Invalid

The final word programming has 11 characters and cannot fit within a width of 10. Therefore, this input is a useful test for invalid word-width handling.

A valid test should use a width large enough to hold the longest individual word. For example, use \(W=12\).

The program must calculate the best arrangement rather than assuming that the above line arrangement is optimal.

10.1 Intermediate DP State Evidence

The program should display a table similar to the following:

Starting word index	Best ending index	Minimum remaining penalty
6	7	0
5	7	Calculated by program
4	6	Calculated by program
3	5	Calculated by program
2	4	Calculated by program
1	3	Calculated by program
0	2	Final minimum penalty

The values must be taken from the actual program execution. The table demonstrates how smaller DP states are used to calculate larger states.

10.2 Requirement Change – Different Line Widths

The same words must be tested using at least three widths.

Example test plan:

Test	Line width	Prediction before execution
Case 1	12	Initial arrangement and penalty
Case 2	15	Some words may fit together; arrangement may change
Case 3	20	More words may fit per line; penalty is expected to decrease or remain unchanged

Interpretation: Increasing the line width gives the algorithm more possible line arrangements. It cannot increase the minimum penalty under this penalty definition because every arrangement that was possible at the smaller width can be represented with no greater unused-space penalty only if the line cost behaviour is carefully considered. In this particular squared-unused-space model, a wider line can sometimes increase the penalty of a non-final line if the same words remain together. Therefore, the exact direction of change must be determined by executing the DP algorithm; do not assume that the penalty always decreases.

10.3 Boundary Test

A useful boundary test is a single word whose length is exactly equal to the line width.

Example:

Input	Value
Words	ALGORITHM
Width	9
Used width	9
Unused spaces	0

Expected behaviour:

- The word should fit exactly.
- The unused spaces should be zero.
- The final line penalty should be zero.
- The program should not create an extra empty line.

This test checks whether the implementation correctly handles exact-width lines and array boundaries.

10.4 Task 1 Conclusion

The Word Wrap problem demonstrates how Dynamic Programming can construct a minimum-penalty document layout. The DP state stores the best remaining cost, while the reconstruction information identifies the words placed on each line. Testing different widths and boundary cases helps verify that the implementation is correct.

TASK 2 – GREEDY VS DYNAMIC PROGRAMMING

KNAPSACK PROBLEM

11. Problem Description

A logistics company has a vehicle with limited carrying capacity. Each available item has a weight and a value. The objective is to select items so that the total value is maximised without exceeding the vehicle's capacity.

The assignment requires two solutions:

- Solution A: Fractional Knapsack using a Greedy Algorithm.
- Solution B: 0/1 Knapsack using Dynamic Programming.

11.1 Input

Each item contains:

- Item ID.
- Weight.
- Value.

The vehicle has a fixed maximum capacity.

11.2 Output

Both implementations must report:

1. Selected items.
2. Selected fraction, where applicable.
3. Total weight.
4. Total value.
5. Computational work.
6. Memory requirement.
7. Comparison of solution quality.

11.3 Difference Between the Two Formulations

Fractional Knapsack: An item can be divided. If the vehicle has space for only part of an item, that fraction can be selected.

0/1 Knapsack: An item must either be selected completely or rejected. Partial selection is not allowed.

11.4 Value-to-Weight Ratio

The Greedy Algorithm uses the ratio:

[R_i=frac{Value_i}{Weight_i}]

Items are arranged in descending order of this ratio.

An item with a higher ratio provides more value per unit of weight.

FRACTIONAL KNAPSACK IMPLEMENTATION

12. Solution A – Fractional Knapsack

Fractional Knapsack uses a Greedy strategy. It selects items based on the highest value-to-weight ratio.

12.1 Sample Dataset

Vehicle capacity:

[W=50]

Item	Weight	Value	Value/Weight ratio
A	10	60	6.0
B	20	100	5.0
C	30	120	4.0

The items are already arranged in descending ratio order.

12.2 Greedy Decision Sequence

Step 1: Select Item A.

- Weight = 10
- Value = 60
- Remaining capacity = 40

Step 2: Select Item B.

- Weight = 20
- Value = 100
- Remaining capacity = 20

Step 3: Item C has weight 30, but only 20 units of capacity remain. Therefore, select \(20/30\) of Item C.

[text{Fraction selected}=frac{20}{30}]

[text{Selected value}=120 times frac{20}{30}=80]

12.3 Final Result

Item	Selected amount	Weight used	Value obtained
A	1	10	60
B	1	20	100
C	2/3	20	80
Total		50	240

Illustrative optimum

Total weight = 50

Total value = 240

12.4 Fractional Knapsack Pseudocode

```
ALGORITHM FractionalKnapsack(items, capacity)
INPUT:
   Items with weight and value
   Vehicle capacity
OUTPUT:
   Selected items and maximum value
BEGIN
   Calculate value-to-weight ratio for each item
   Sort items by descending ratio
   totalValue ← 0
   FOR each item in sorted items DO
      IF capacity = 0 THEN
         BREAK
      END IF
      IF item.weight ≤ capacity THEN
         Select complete item
         capacity ← capacity - item.weight
         totalValue ← totalValue + item.value
      ELSE
         fraction ← capacity / item.weight
         Select fraction of item
         totalValue ← totalValue +
                  fraction × item.value
         capacity ← 0
      END IF
   END FOR
   RETURN selected items and totalValue
END
```

The algorithm sorts the items and then scans them once. Its main computational work comes from sorting and checking the items.

0/1 KNAPSACK USING DYNAMIC PROGRAMMING

13. Solution B – 0/1 Knapsack

In 0/1 Knapsack, each item can be selected only once. The item cannot be divided into fractions.

Dynamic Programming compares two possibilities for every item:

1. Exclude the item.
2. Include the item, if its weight does not exceed the remaining capacity.

Let:

[DP[i][w]]

represent the maximum value obtainable using the first \(i\) items and a capacity of \(w\).

13.1 Base Condition

[DP[0][w]=0]

No value can be obtained when there are no items.

[DP[i][0]=0]

No item can be selected when the capacity is zero.

13.2 Recurrence Relation

If the current item's weight is greater than the available capacity:

[DP[i][w]=DP[i-1][w]]

Otherwise:

[DP[i][w]=max left(DP[i-1][w], Value_i+DP[i-1][w-Weight_i] right)]

The first option excludes the item. The second option includes it.

13.3 Greedy Counterexample Dataset

Vehicle capacity:

[W=50]

Item	Weight	Value	Ratio
A	10	60	6.0
B	20	100	5.0
C	30	120	4.0

The Greedy ratio order selects A first and B second. The remaining capacity is 20, so C cannot be selected.

Greedy result:

[60+100=160]

However, Dynamic Programming can select B and C:

[20+30=50]

[100+120=220]

Therefore:

[GreedyResult neq DPOptimum]

[160 neq220]

13.4 First Greedy Commitment That Causes Failure

The first greedy commitment is selecting Item A because it has the highest value-to-weight ratio.

This decision uses 10 units of capacity. It prevents the combination of Items B and C from being selected together because their combined weight already equals the vehicle capacity.

The locally best ratio does not produce the globally best 0/1 selection.

0/1 KNAPSACK PSEUDOCODE AND COMPARISON

14. 0/1 Knapsack Pseudocode

ALGORITHM Knapsack01(items, capacity)

INPUT:

Item weights and values

Vehicle capacity

OUTPUT:

Maximum value and selected items

```
BEGIN
  n ← number of items
  Create DP table of size (n+1) × (capacity+1)

  FOR i ← 0 TO n DO
    DP[i][0] ← 0
  END FOR
```

```
    FOR w ← 0 TO capacity DO
        DP[0][w] ← 0
    END FOR
    FOR i ← 1 TO n DO
        FOR w ← 1 TO capacity DO
            IF weight[i] ≤ w THEN
                include ← value[i] +
                        DP[i-1][w-weight[i]]
                exclude ← DP[i-1][w]
                DP[i][w] ← MAX(include, exclude)
            ELSE
                DP[i][w] ← DP[i-1][w]
            END IF
        END FOR
    END FOR
    Reconstruct selected items from DP table
    RETURN DP[n][capacity] and selected items
END
```

14.1 Computational Comparison

Feature	**Fractional Knapsack**	**0/1 Knapsack**
Strategy	Greedy	Dynamic Programming
Item selection	Complete or partial	Complete or rejected
Main decision	Highest value-to-weight ratio	Include or exclude
Sorting	Required for ratio-based approach	Not required
Typical time complexity	\(O(n\log n)\)	\(O(nW)\)
Memory requirement	Usually \(O(n)\) for item storage	\(O(nW)\) for full DP table

Optimality	Optimal for fractional formulation	Optimal for the given integer-capacity DP formulation

The (O(nW)) complexity is pseudo-polynomial because it depends on the numerical capacity (W), not only on the number of items.

14.2 Instrumentation

The program can count:

- Number of ratio calculations.
- Number of sorting comparisons, if measured by the sorting implementation.
- Number of item-selection decisions.
- Number of DP table cells evaluated.
- Number of items selected.

For a full 0/1 DP table, the number of table cells is approximately:

[(n+1)(W+1)]

This gives a direct indication of the work performed.

CHANGED REQUIREMENT AND TASK 2 EVALUATION

15. Engineering Comparison

The two algorithms solve different versions of the Knapsack problem. Fractional Knapsack allows partial items, whereas 0/1 Knapsack does not.

The Greedy solution is suitable when partial selection is allowed and a quick optimal result is required. Dynamic Programming is appropriate when items must be selected completely and the exact maximum value is required.

15.1 Requirement Change

Management states:

“The decision must be returned extremely quickly. A near-optimal result may be acceptable.”

This changes the selection criteria. The system no longer needs to guarantee the exact mathematical optimum in every case. Response time becomes more important.

For a large 0/1 Knapsack instance, the DP table may require a large number of states. A Greedy ratio-based method can generate a fast candidate solution, but it may not be optimal.

The implementation should compare the value obtained by the fast method with the DP optimum on smaller test cases where DP is practical. This helps estimate the quality of the fast strategy.

15.2 Engineering Decision Table

Requirement	**Suitable strategy**	**Reason**

Partial items are allowed	Fractional Knapsack	Ratio-based Greedy is optimal
Items must be complete and exact optimum is needed	0/1 Knapsack DP	Evaluates include/exclude choices
Very fast result is required	Greedy-based approximation or heuristic to investigate	Can reduce computation, but may lose optimality
Memory is limited	A memory-optimised DP or heuristic to investigate	Full DP table can be expensive

15.3 Interpretation of the Counterexample

The counterexample demonstrates that the value-to-weight ratio alone is not sufficient for 0/1 Knapsack. Item A has the highest ratio, but selecting it prevents the better combination of Items B and C.

The DP solution examines the include and exclude decisions and identifies the combination with the highest total value. This proves that the two formulations require different optimisation strategies.

15.4 Task 2 Conclusion

Fractional Knapsack demonstrates the effectiveness of a Greedy strategy when partial selection is allowed. The 0/1 Knapsack counterexample demonstrates the limitation of applying the same ratio-based strategy when items cannot be divided. Dynamic Programming provides the exact optimal solution for the tested 0/1 instance, while the Greedy method provides a faster decision process with no general optimality guarantee for that formulation.

TASK 3 – ROUTING ENGINE UNDER CHANGING NETWORK CONDITIONS

GRAPH REPRESENTATION AND DIJKSTRA'S ALGORITHM

16. Problem Description

A logistics platform requires a routing system that calculates the shortest route between locations. The network is represented as a weighted directed graph.

Each vertex represents a location, and each directed edge represents a connection between two locations. The edge weight represents the travel cost or distance.

The assignment requires implementing:

1. Dijkstra's Algorithm for normal routing.
2. Bellman-Ford Algorithm after introducing a negative edge.
3. Negative-weight cycle detection.
4. Floyd's Algorithm for all-pairs shortest paths.
5. Warshall's Algorithm for reachability.

16.1 Graph Input

The network contains eight vertices:

[V={A,B,C,D,E,F,G,H}]

Use the following directed edges for the initial graph.

Edge	Starting vertex	Ending vertex	Weight
1	A	B	2
2	A	C	5
3	B	D	6
4	C	D	1
5	D	E	2
6	E	F	2
7	F	G	1
8	G	H	2
9	B	E	10
10	C	F	8
11	D	G	7
12	E	H	10

The graph has eight vertices and twelve directed edges, satisfying the assignment requirement.

16.2 Adjacency List Representation

The graph can be stored using an adjacency list. Each vertex stores its outgoing neighbours and corresponding weights.

Example:

A → (B,2), (C,5)

B → (D,6), (E,10)

C → (D,1), (F,8)

D → (E,2), (G,7)

E → (F,2), (H,10)

F → (G,1)

G → (H,2)

H → None

An adjacency list is useful because it stores only the edges that actually exist in the graph.

16.3 Dijkstra's Algorithm

Dijkstra's Algorithm calculates shortest paths from a selected source vertex when all edge weights are non-negative.

The algorithm maintains:

- distance[v]: shortest currently known distance from the source to vertex \(v\).
- predecessor[v]: previous vertex on the currently best-known path.
- visited[v]: whether the vertex has been finalised.

Initially, the source distance is zero, and all other distances are infinity.

The algorithm repeatedly selects the unfinalised vertex with the smallest distance and relaxes its outgoing edges.

16.4 Relaxation

For an edge (urightarrow v) with weight \(w\), the algorithm checks:

[distance[u]+w<distance[v]]

If the condition is true, the distance and predecessor are updated.

This process continues until all reachable vertices have been finalised.

PAGE 12 – DIJKSTRA IMPLEMENTATION AND NORMAL ROUTING

17. Dijkstra's Pseudocode

ALGORITHM Dijkstra(Graph, source)

INPUT:

Weighted directed graph

Source vertex

OUTPUT:

Shortest distances

Predecessors

Finalisation order

BEGIN

FOR each vertex v DO

distance[v] ← INFINITY

predecessor[v] ← NIL

```
    visited[v] ← FALSE
  END FOR
  distance[source] ←
  WHILE an unvisited reachable vertex exists D
    u ← unvisited vertex with minimum distance
    visited[u] ← TRUE
    Record u in finalisation order
    FOR each edge u → v with weight w DO
      IF visited[v] = FALSE AND
        distance[u] + w < distance[v] THEN
        distance[v] ← distance[u] + w
        predecessor[v] ←
      END IF
    END FOR
  END WHILE
  RETURN distance, predecessor and finalisation order
END
```

17.1 Normal Routing from A

Using the initial graph, the shortest distances from source A are:

Vertex	Shortest distance	Predecessor	Shortest path
A	0	NIL	A
B	2	A	A → B
C	5	A	A → C
D	6	C	A → C → D
E	8	D	A → C → D → E
F	10	E	A → C → D → E → F
G	11	F	A → C → D → E → F → G
H	13	G	A → C → D → E → F → G → H

17.2 Explanation of the Result

The shortest path from A to B is the direct edge with cost 2.

The shortest path from A to D is through C:

[A rightarrow C rightarrow D]

[5+1=6]

The shortest path from A to H is:

[A rightarrow C rightarrow D rightarrow E rightarrow Frightarrow G rightarrow H]

[5+1+2+2+1+2=13]

The predecessor array stores the previous vertex at each step. It allows the program to reconstruct the actual route rather than displaying only the numerical distance.

17.3 Route Reconstruction

To reconstruct a path, begin at the destination and repeatedly follow its predecessor until the source is reached. Reverse the collected sequence to obtain the correct route.

For example, the predecessor sequence for H is:

H ← G ← F ← E ← D ← C ← A

After reversing:

A → C → D → E → F → G → H

This is the route that should be displayed by the program.

INTRODUCING A NEGATIVE EDGE AND BELLMAN-FORD

18. Stage B – Introducing a Negative Edge

The assignment requires modifying the graph by introducing a carefully selected negative-weight edge.

A negative edge represents a connection whose cost is negative under the chosen mathematical model. It may represent a discount, credit or adjustment in a cost network.

Modify the initial graph by adding:

[C rightarrow B=-4]

The modified graph now contains the following important connections:

Edge	Weight
A → B	2
A → C	5
C → B	-4

C → D	1
B → D	6

18.1 Prediction Before Execution

Before running the modified graph, the prediction is:

1. Dijkstra's Algorithm is no longer guaranteed to produce correct shortest paths because the graph contains a negative edge.
2. Vertex B may be finalised too early using the direct route A → B.
3. Later, reaching C may reveal a better route from A to B through the negative edge.
4. Bellman-Ford should identify the shorter route correctly because it repeatedly relaxes all edges.

The original direct path to B has cost:

[A rightarrow B=2]

The alternative path is:

[Arightarrow C rightarrow B]

[5+(-4)=1]

Therefore, the alternative route is shorter.

18.2 Why Dijkstra Can Fail

Dijkstra's Algorithm assumes that once a vertex is finalised, no later path can reduce its distance.

A negative edge can violate this assumption. A vertex that was previously considered final may later receive a smaller distance through another route.

In this example, B may be finalised with distance 2 before C is processed. However, the path through C has cost 1.

This demonstrates why Dijkstra's standard correctness guarantee requires non-negative edge weights.

18.3 Bellman-Ford Algorithm

Bellman-Ford supports negative edge weights, provided that no reachable negative-weight cycle makes shortest paths undefined.

It relaxes all edges repeatedly for at most:

[|V|-1]

passes.

For a graph with eight vertices, the algorithm requires at most seven complete relaxation passes, followed by an additional pass to check for a reachable negative-weight cycle.

BELLMAN-FORD AND NEGATIVE-CYCLE DETECTION

19. Bellman-Ford Pseudocode

ALGORITHM BellmanFord(Graph, source)

INPUT:

Weighted directed graph

Source vertex

OUTPUT:

Shortest distances

Predecessors

Negative-cycle status

```
BEGIN
    FOR each vertex v DO
        distance[v] ← INFINITY
        predecessor[v] ← NIL
    END FOR
    distance[source] ← 0
    FOR i ← 1 TO |V|-1 DO
        changed ← FALSE
        FOR each edge (u, v, w) DO
            IF distance[u] ≠ INFINITY AND
               distance[u] + w < distance[v] THEN
                distance[v] ← distance[u] + w
                predecessor[v] ← u
                changed ← TRUE
            END IF
        END FOR
        IF changed = FALSE THEN
            BREAK
        END IF
    END FOR
```

```
    FOR each edge (u, v, w) D
        IF distance[u] ≠ INFINITY AND
            distance[u] + w < distance[v] THEN
            RETURN "Reachable negative cycle detected"
        END IF
    END FOR
    RETURN distance, predecessor
END
```

19.1 Bellman-Ford Result for the Modified Graph

The distance to B becomes:

$$ distance[B]=1 $$

The updated route is:

$$ A\rightarrow C\rightarrow B $$

The distance to D is still:

$$ distance[D]=6 $$

because the direct route through C is:

$$ A\rightarrow C\rightarrow D=5+1=6 $$

The algorithm should report the final distances and reconstructed paths from its execution.

19.2 Stage C – Production Failure

Now introduce another edge:

$$ H\rightarrow C=-20 $$

There is already a path from C to H:

$$ C\rightarrow D\rightarrow E\rightarrow F \rightarrow G\rightarrow H $$

Its cost is:

$$ 1+2+2+1+2=8 $$

The cycle cost is:

$$ 8+(-20)=-12 $$

Therefore, the graph contains a reachable negative-weight cycle.

19.3 How the Program Detects the Cycle

Bellman-Ford normally performs $(|V|-1)$ relaxation passes. After these passes, it checks every edge one more time.

If any edge can still reduce a distance, a reachable negative-weight cycle exists.

The program must display a clear message such as:

ERROR: Reachable negative-weight cycle detected.

Shortest-path results are not valid.

It must not present the calculated distances as final shortest paths.

PAGE 15 – FLOYD'S AND WARSHALL'S ALGORITHMS

20. Stage D – All-Pairs Shortest Paths

The analytics team now requires the shortest-path cost between every pair of vertices.

Dijkstra and Bellman-Ford calculate shortest paths from a selected source. Floyd's Algorithm calculates shortest-path distances between all pairs of vertices.

20.1 Floyd's Algorithm

Floyd's Algorithm uses a distance matrix.

Let:

[D[i][j]]

represent the shortest currently known distance from vertex (i) to vertex (j).

Initially:

- The distance from a vertex to itself is zero.
- The distance for a direct edge is its weight.
- The distance for a missing edge is infinity.

The algorithm checks whether travelling through an intermediate vertex \(k\) provides a shorter route.

The recurrence is:

[D[i][j]=min(D[i][j],D[i][k]+D[k][j])]

20.2 Floyd's Pseudocode

ALGORITHM Floyd(Graph)

INPUT:

Weighted adjacency matrix

OUTPUT:

All-pairs shortest-distance matrix

BEGIN

D ← Copy of adjacency matrix

```
    FOR k ← 0 TO n-1 DO
        FOR i ← 0 TO n-1 DO
            FOR j ← 0 TO n-1 DO
                IF D[i][k] + D[k][j] < D[i][j] THEN
                    D[i][j] ← D[i][k] + D[k][j]
                END IF
            END FOR
        END FOR
    END FOR
    RETURN D
END
```

20.3 Warshall's Algorithm

Warshall's Algorithm determines whether one vertex is reachable from another.

It uses a Boolean matrix instead of a distance matrix.

[R[i][j]=TRUE]

means that a path exists from vertex \(i\) to vertex \(j\).

The recurrence is:

[R[i][j]=R[i][j] OR (R[i][k] AND R[k][j])]

The algorithm checks whether a path from \(i\) to \(j\) becomes possible through intermediate vertex \(k\).

20.4 Warshall's Pseudocode

ALGORITHM Warshall(Graph)

INPUT:

Boolean adjacency matrix

OUTPUT:

Transitive closure matrix

```
BEGIN
    R ← Copy of adjacency matrix
    FOR i ← 0 TO n-1 DO
        R[i][i] ← TRUE
    END FOR
```

```
    FOR k ← 0 TO n-1 DO
        FOR i ← 0 TO n-1 DO
            FOR j ← 0 TO n-1 DO
                R[i][j] ← R[i][j] OR
                        (R[i][k] AND R[k][j])
            END FOR
        END FOR
    END FOR
    RETURN R
END
```

For the original graph, all vertices are reachable from A, but reachability between other ordered pairs depends on the direction of the edges.

20.5 Task 3 Complexity Comparison

Algorithm	Purpose	Time complexity
Dijkstra with simple array	Single-source shortest path	\(O(V^2+E)\)
Dijkstra with binary heap	Single-source shortest path	\(O((V+E)\log V)\)
Bellman-Ford	Single-source paths with negative edges	\(O(VE)\)
Floyd's Algorithm	All-pairs shortest paths	\(O(V^3)\)
Warshall's Algorithm	All-pairs reachability	\(O(V^3)\)

Important: Run Floyd's Algorithm on a graph without a negative cycle when interpreting shortest-path distances. A negative cycle makes shortest-path costs undefined for affected vertex pairs.

20.6 Task 3 Conclusion

The routing experiment demonstrates that algorithm selection depends on graph conditions. Dijkstra is efficient for non-negative edge weights, while Bellman-Ford supports negative edges and detects reachable negative cycles. Floyd's Algorithm provides all-pairs shortest distances, and Warshall's Algorithm identifies reachability between every pair of vertices.

TASK 4 – SMART MANUFACTURING OPTIMISATION

ASSEMBLY LINE SCHEDULING PROBLEM

21. Problem Description

A manufacturing company operates two parallel assembly lines. Each product must pass through a sequence of processing stations.

At each station, the product can either remain on the current line or transfer to the other line before entering the next station.

Every station has a processing time, and each transfer has an additional time cost.

The objective is to calculate the minimum total production time and identify the exact sequence of assembly-line stations that achieves it.

21.1 Input

The program must accept:

- Processing times for Line 1.
- Processing times for Line 2.
- Transfer times between the two lines.
- Entry times.
- Exit times.

21.2 Output

The program must display:

1. Minimum production time.
2. Selected line at every station.
3. Transfer points.
4. Final optimal route.
5. Intermediate DP decisions.

21.3 Sample Dataset

Consider four processing stations.

Parameter	Line 1	Line 2
Station 1	4	6
Station 2	5	3
Station 3	3	7
Station 4	2	4

Transfer time from Line 1 to Line 2:

Transfer	Time
After Station 1	2

After Station 2	2
After Station 3	1

Transfer time from Line 2 to Line 1:

Transfer	Time
After Station 1	1
After Station 2	3
After Station 3	2

Entry times:

[e_1=2,qquad e_2=4]

Exit times:

[x_1=3,qquad x_2=2]

These values will be used to demonstrate the DP calculation.

21.4 Why Dynamic Programming Is Suitable

The minimum time to reach a station depends on the best time to reach the previous station on either line.

The same partial station sequences are used in multiple possible routes. Storing their minimum costs avoids repeatedly evaluating all possible line combinations.

The problem also has optimal substructure because an optimal route to a station must use an optimal route to the previous station from which it arrives.

ASSEMBLY LINE DP STATE AND RECURRENCE

22. Dynamic Programming State

Let:

[f_1[j]]

represent the minimum time required to reach station \(j\) on Line 1.

Let:

[f_2[j]]

represent the minimum time required to reach station \(j\) on Line 2.

The entry conditions are:

[f_1[1]=e_1+a_{1,1}]

[f_2[1]=e_2+a_{2,1}]

where (a_{1,j}) and (a_{2,j}) represent the processing times at station \(j\).

22.1 Recurrence for Line 1

To reach station (j) on Line 1, there are two possibilities:

1. Stay on Line 1 from station (j-1).
2. Transfer from Line 2 after station (j-1).

Therefore:

[f_1[j]=min begin{cases} f_1[j-1]+a_{1,j} f_2[j-1]+t_{2,j-1}+a_{1,j} end{cases}]

22.2 Recurrence for Line 2

Similarly:

[f_2[j]=min begin{cases} f_2[j-1]+a_{2,j} f_1[j-1]+t_{1,j-1}+a_{2,j} end{cases}]

22.3 Final Minimum Time

After processing the last station, the product must leave through one of the two lines.

[T=\min(f_1[n]+x_1, f_2[n]+x_2)]

The algorithm must also store the selected predecessor line at every station. This information is necessary to reconstruct the route.

22.4 DP Decision Table

For each station, the program should display:

Station	Reach Line 1 by staying	Reach Line 1 by transfer	Selected	Reach Line 2 by staying	Reach Line 2 by transfer	Selected
2	Calculated	Calculated	Line	Calculated	Calculated	Line
3	Calculated	Calculated	Line	Calculated	Calculated	Line
4	Calculated	Calculated	Line	Calculated	Calculated	Line

The actual values should be filled from the program's execution.

ASSEMBLY LINE WORKED EXAMPLE AND ROUTE RECONSTRUCTION

23. DP Calculation

For the sample dataset:

[f_1[1]=2+4=6]

[f_2[1]=4+6=10]

At Station 2, Line 1:

- Stay on Line 1: (6+5=11)

- Transfer from Line 2: (10+1+5=16)

Therefore:

[f_1[2]=11]

At Station 2, Line 2:

- Stay on Line 2: (10+3=13)
- Transfer from Line 1: (6+2+3=11)

Therefore:

[f_2[2]=11]

At Station 3, Line 1:

- Stay: (11+3=14)
- Transfer from Line 2: (11+3+3=17)

Therefore:

[f_1[3]=14]

At Station 3, Line 2:

- Stay: (11+7=18)
- Transfer from Line 1: (11+2+7=20)

Therefore:

[f_2[3]=18]

At Station 4, Line 1:

- Stay: (14+2=16)
- Transfer from Line 2: (18+2+2=22)

Therefore:

[f_1[4]=16]

At Station 4, Line 2:

- Stay: \(18+4=22\)
- Transfer from Line 1: \(14+1+4=19\)

Therefore:

[f_2[4]=19]

23.1 Final Result

Including exit times:

[T_1=16+3=19]

[T_2=19+2=21]

Therefore:

[T=min(19,21)=19]

Illustrative minimum time

Minimum production time = 19 time units.

The optimal route is:

Entry

↓

Line 1 – Station 1

↓

Line 1 – Station 2

↓

Line 1 – Station 3

↓

Line 1 – Station 4

↓

Exit

23.2 Route Reconstruction

The program should store whether each station was reached by staying on the same line or transferring from the other line.

After selecting the best final line, it traces the stored decisions backwards from the final station to the first station.

This produces the complete route.

23.3 Assembly Line Pseudocode

ALGORITHM AssemblyLine(a1, a2, t1, t2, e1, e2, x1, x2)

INPUT:

Processing times

Transfer times

Entry times

Exit times

OUTPUT:

Minimum production time

Optimal line sequenc

BEGIN

f1[1] ← e1 + a1[1]

f2[1] ← e2 + a2[1]

FOR j ← 2 TO n DO

stay1 ← f1[j-1] + a1[j]

transfer1 ← f2[j-1] + t2[j-1] + a1[j]

f1[j] ← MIN(stay1, transfer1)

Store decision for Line 1

stay2 ← f2[j-1] + a2[j]

transfer2 ← f1[j-1] + t1[j-1] + a2[j]

f2[j] ← MIN(stay2, transfer2)

Store decision for Line 2

END FOR

final1 ← f1[n] + x1

final2 ← f2[n] + x2

Select minimum final cost

Reconstruct line sequence

RETURN minimum time and route

END

ASSEMBLY LINE SCHEDULING

24. Increase a Transfer Time

Change the transfer time from Line 1 to Line 2 after Station 2.

- Original: (t_{1,2}=2)
- Modified: (t_{1,2}=20)

Prediction:

1. Transferring to Line 2 becomes more expensive.
2. The algorithm may choose to stay on Line 1.

3. Minimum production time cannot decrease.
4. The route changes only if another route becomes better.

Measurement	Original	Modified
Transfer time	2	20
Minimum time	19	Actual output
Optimal route	Actual route	Actual route

The DP algorithm compares staying and transferring to select the minimum-cost route.

25. Production Disruption Challenge

Increase the processing time of Line 1 at Station 3.

- Original: (a_{1,3}=3)
- Modified: (a_{1,3}=15)

The increased time may make Line 2 a better choice. The program must be executed to identify the new optimal route.

Measurement	Normal	Disrupted
Line 1, Station 3	3	15
Minimum production time	19	Actual output
Selected line	Actual route	Actual route
Transfer introduced	Actual route	Yes/No

The algorithm compares the cost of staying and transferring. It selects the route with the lowest total production time.

26. Extension to Three Assembly Lines

For two lines, the DP states are:

\[f_1[j],\quad f_2[j] \]

For three lines, the states become:

\[f_1[j],\quad f_2[j],\quad f_3[j] \]

Each state stores the minimum time to reach station \(j\) on a particular line.

The algorithm must compare staying on the current line and transferring from the other available lines. It must also store the previous line to reconstruct the final route.

Feature	Two Lines	Three Lines

DP states	2	3
Previous lines	2	3
Transition choices	Up to 2	Up to 3
Route reconstruction	Backtracking	Backtracking

27. Task 4 Conclusion

Assembly Line Scheduling uses Dynamic Programming to find the minimum production time and optimal route. Changing transfer and processing times helps test how the algorithm responds to production changes.

TASK 5 – THREE GREEDY ENGINEERS, ONE NETWORK

28. Minimum Spanning Tree Problem

A company wants to connect all its facilities with minimum installation cost. The network is represented using a connected, undirected, weighted graph.

- Vertices represent facilities.
- Edges represent connections.
- Weights represent installation costs.

A Minimum Spanning Tree (MST) connects all vertices without cycles using exactly:

$$ V-1 $$

edges.

28.1 Algorithms

1. Kruskal's Algorithm.
2. Prim's Algorithm.
3. Borůvka's Algorithm.

28.2 Sample Graph

Vertices:

$$ V=\{A,B,C,D,E,F,G,H\} $$

Edge	Connection	Weight
1	A–B	3
2	A–C	3
3	B–C	1

4	B–D	5
5	C–D	4
6	C–E	6
7	D–E	2
8	D–F	7
9	E–F	3
10	E–G	5
11	F–G	2
12	G–H	4
13	F–H	6

A valid MST must connect all vertices, contain no cycles and have seven edges with minimum total weight.

29. Kruskal's Algorithm

Kruskal's Algorithm selects edges in increasing order of weight. It adds an edge only if it connects two different components. Otherwise, it rejects the edge to avoid a cycle.

A Disjoint Set Union (DSU) structure is used to manage components.

Pseudocode:

ALGORITHM Kruskal(Graph)

Sort all edges by increasing weight

Create separate components for all vertices

MST = empty

totalCost =

FOR each edge (u, v, weight):

  IF FIND(u) != FIND(v):

    Add edge to MST

    totalCost = totalCost + weight

    UNION(u, v)

  ELSE:

    Reject edge

IF MST contains V-1 edges:

BREAK

RETURN MST and totalCost

29.1 Edge Selection

Edge	Weight	Decision
B–C	1	Select
D–E	2	Select
F–G	2	Select
A–B	3	Select
E–F	3	Select
A–C	3	Reject
C–D	4	Select
G–H	4	Select

29.2 Final MST

Selected Edge	Weight
B–C	1
D–E	2
F–G	2
A–B	3
E–F	3
C–D	4
G–H	4
Total	19

The MST connects all eight vertices using seven edges.

30. Prim's Algorithm

Prim's Algorithm starts from one vertex and repeatedly selects the minimum-weight edge connecting the current tree to a vertex outside it.

Pseudocode:

ALGORITHM Prim(Graph, start)

Set all keys to INFINITY

Set all parents to NIL

Mark all vertices as not included

Set key[start] = 0

REPEAT V times:

    Select the vertex u with minimum key

    Mark u as included

    IF parent[u] is not NIL:

        Add edge (parent[u], u) to MST

    FOR each neighbour v of u:

        IF v is not included AND weight(u,v) < key[v]:

            key[v] = weight(u,v)

            parent[v] = u

RETURN MST and totalCost

30.1 Example Starting from A

Step	Selected Edge	Weight
1	Start at A	0
2	A–B	3
3	B–C	1
4	C–D	4
5	D–E	2
6	E–F	3
7	F–G	2
8	G–H	4

Total cost:

$$ 3+1+4+2+3+2+4=19 $$

30.2 Kruskal and Prim Comparison

Feature	Kruskal	Prim
Starting point	No fixed vertex	Selected vertex
Growth	Combines components	Grows one tree
Edge selection	Smallest safe edge	Smallest outgoing edge
Data structure	DSU and sorted edges	Priority queue or key array
Cycle prevention	DSU	Included-vertex check

31. Borůvka's Algorithm

Borůvka's Algorithm finds the cheapest outgoing edge from every connected component and combines the components. It repeats until only one component remains.

Pseudocode:

```
ALGORITHM Boruvka(Graph)

Create a separate component for every vertex

MST = empty

totalCost = 0

WHILE more than one component exists:

   Find the cheapest outgoing edge

   for every component

   FOR each selected edge (u, v, weight):

      IF FIND(u) != FIND(v):

         Add edge to MST

         totalCost = totalCost + weight

         UNION(u, v)

RETURN MST and totalCost
```

31.1 Working Process

Phase	Description
Initial	Every vertex is separate
Phase 1	Cheapest edges connect components
Phase 2	Larger components find outgoing edges

Phase 3	Remaining components are combined
Final	All vertices form one component

The program must display the actual selected edges in every phase. DSU prevents duplicate selections and cycles.

32. MST Validation and Comparison

Measurement	Kruskal	Prim	Borůvka
Main operation	Sort edges	Select minimum edge	Find cheapest edge per component
Growth	Combines components	Adds one vertex	Combines components in phases
Cycle handling	DSU	Included-vertex check	DSU
Output	Selected edges	Selected edges	Edges by phase

32.1 Complexity Comparison

For \(V\) vertices and \(E\) edges:

Algorithm	Time Complexity
Kruskal	\(O(E\log E)\)
Prim with binary heap	\(O(E\log V)\)
Prim with matrix	\(O(V^2)\)
Borůvka	\(O(E\log V)\)

32.2 Multiple MST Experiment

Modify the triangle edge weights:

Edge	Modified Weight
A–B	3
A–C	3
B–C	3

Possible MSTs include:

- A–B and B–C.
- A–C and B–C.

- A–B and A–C.

Different algorithms may select different edges because of equal weights. All are correct if they connect every vertex without cycles and have the minimum total cost.

32.3 Industry Decisions

Sparse Network: Kruskal or Prim with an adjacency list can be suitable.

Dense Network: Prim with an adjacency matrix can be useful.

Large Distributed Network: Borůvka provides opportunities for parallel processing because components can find their cheapest outgoing edges in each phase.

32.4 Task 5 Conclusion

Kruskal, Prim and Borůvka construct minimum spanning trees using Greedy decisions. They differ in how they select edges and grow the network. Multiple MSTs may exist when edges have equal weights.

TASK 6 – LAST-MILE DELIVERY ROUTE OPTIMISATION

33. Travelling Salesperson Problem

A logistics company must deliver products from a central hub to several locations and return to the hub with minimum travel cost.

The vehicle must:

1. Start from the hub.
2. Visit every location exactly once.
3. Return to the hub.
4. Minimise total travel cost.

This problem is called the Travelling Salesperson Problem (TSP).

33.1 Input and Output

Input:

- One central hub.
- At least six delivery locations.
- A travel-cost matrix.

Output:

- Minimum travel cost.
- Complete delivery route.
- Order of locations.
- Return to the hub.

33.2 Constraints

- Start and finish at the hub.
- Visit every delivery location exactly once.
- Minimise total travel cost.

33.3 DP State

Knowing only the current location is insufficient because different routes may reach the same location after visiting different places.

Therefore, the algorithm stores the visited locations and current location using:

$$ DP[S][j] $$

34. TSP DP State and Recurrence

The state $DP[S][j]$ represents the minimum cost of starting at the hub, visiting all locations in set S, and ending at location j.

34.1 Base Condition

$$ DP[\{j\}][j]=cost[0][j] $$

This represents travelling directly from the hub to location j.

34.2 State Transition

$$ DP[S][j]=\min_{i\in S,\ i\neq j} (DP[S-\{j\}][i]+cost[i][j]) $$

The algorithm checks all possible previous locations and selects the minimum cost.

34.3 Final Return to Hub

$$ \min_{j\in V}(DP[V][j]+cost[j][0]) $$

The vehicle returns to the hub after visiting all delivery locations.

Component	Definition
State	$DP[S][j]$
Base	Direct travel from hub
Transition	Minimum cost from previous location
Final condition	Return to hub
Reconstruction	Store previous location

35. Sample Travel-Cost Matrix

Use one hub and six delivery locations.

Location	A (Hub)	B	C	D	E	F	G
A (Hub)	0	10	15	20	10	25	18
B	10	0	35	25	17	28	20
C	15	35	0	30	22	18	26
D	20	25	30	0	16	24	19
E	10	17	22	16	0	14	21
F	25	28	18	24	14	0	12
G	18	20	26	19	21	12	0

35.1 TSP Pseudocode

```
ALGORITHM TSP_DP(cost)
Create DP table and parent table
FOR each location j:
   DP[{j}][j] = cost[hub][j]
FOR each subset S:
   FOR each location j in S:
      FOR each location i in S, i != j:
         candidate = DP[S-{j}][i] + cost[i][j]
         IF candidate < DP[S][j]:
            DP[S][j] = candidate
            parent[S][j] = i
Find minimum cost of returning to hub
Reconstruct route using parent table
RETURN minimum cost and route
```

35.2 Minimum Route

One minimum route for the sample matrix is:

A → B → E → D → G → F → C → A

Route Segment	Cost
A → B	10

B → E	17
E → D	16
D → G	19
G → F	12
F → C	18
C → A	15
Total	107

$$ 10+17+16+19+12+18+15=107 $$

Minimum travel cost = 107.

The parent table stores previous locations and helps reconstruct the complete route.

36. TSP Validation

Validate the DP solution using four delivery locations. There are:

$$ 4!=24 $$

possible delivery orders.

Using locations B, C, D and E, the minimum route is:

A → B → D → E → C → A

Route Segment	Cost
A → B	10
B → D	25
D → E	16
E → C	22
C → A	15
Total	88

Method	Minimum Cost
Exhaustive Enumeration	88
Dynamic Programming	88

Both methods produce the same minimum cost. This helps validate the DP implementation.

37. TSP State-Space Growth

For \(n\) delivery locations, the number of subsets is:

\[2^n \]

Held-Karp DP complexity:

- Time: \(O(n^2 2^n)\)
- Memory: \(O(n2^n)\)

37.1 Experimental Table

Delivery Locations	Subsets \(2^n\)	DP States	Execution Time
4	16	Actual count	Actual time
5	32	Actual count	Actual time
6	64	Actual count	Actual time
7	128	Actual count	Actual time
8	256	Actual count	Actual time

As the number of locations increases, the number of subsets doubles. DP avoids repeated calculations but still requires processing many states.

38. Business Requirement Change

Management now has 200 delivery locations and requires a route within a few seconds. A near-optimal route is acceptable.

Exact DP becomes impractical because its time complexity is:

\[O(n^2 2^n) \]

For 200 locations, the number of subsets is \(2^{200}\), which is too large for ordinary exact DP execution.

38.1 Alternative Strategies

The company can investigate:

- Nearest Neighbour.
- Local Search.
- 2-opt improvement.
- Genetic Algorithms.
- Other approximation methods.

Exact DP	Heuristic Methods

Guarantees an optimum	May not guarantee an optimum
Exponential growth	Often faster
High memory for large inputs	Often requires less memory
Suitable for smaller problems	Useful for larger problems

The company can accept a slightly higher travel cost to obtain a good route quickly.

39. Technical Reflection

Dynamic Programming reduces repeated calculations by storing previously calculated states. However, TSP still has exponentially many subsets. Its time complexity is \(O(n^2 2^n)\), and its memory complexity is \(O(n2^n)\). Therefore, DP is useful for smaller problems but is not practical for 200 delivery locations. A heuristic or approximation method should be considered when a fast near-optimal route is acceptable.

40. Overall Comparison of Six Tasks

Task	**Algorithm**	**Main Objective**
1	Word Wrap DP	Minimum layout penalty
2	Fractional and 0/1 Knapsack	Compare Greedy and DP
3	Dijkstra, Bellman-Ford, Floyd and Warshall	Routing and reachability
4	Assembly Line Scheduling DP	Minimum production time
5	Kruskal, Prim and Borůvka	Minimum-cost network
6	TSP DP	Minimum delivery route

41. Mandatory Submission Evidence

Evidence	**Requirement**
Source code	Complete program
Test input	Dataset used
Expected behaviour	Prediction before execution
Actual output	Program result
Intermediate evidence	DP states, edges or route decisions
Boundary test	Test unusual or invalid cases
Interpretation	Explain the result

Screenshots should show inputs, outputs and intermediate algorithm results.

42. Software Engineering Expectations

1. Use meaningful variable and function names.
2. Separate algorithm logic and input/output.
3. Select suitable data structures.
4. Handle boundary conditions and invalid inputs.
5. State input assumptions.
6. Use reproducible test cases.
7. Handle exceptional conditions.
8. Display intermediate results.

43. Final Conclusion

This assignment demonstrates how Dynamic Programming and Greedy Algorithms solve practical optimisation problems. Word Wrap and Assembly Line Scheduling use DP to find optimal solutions. Knapsack compares Greedy and DP approaches. Graph algorithms solve routing and reachability problems. Kruskal, Prim and Borůvka construct minimum spanning trees. TSP demonstrates how DP reduces repeated calculations but still faces exponential growth.